Лекция 5. Потоки: Форд-Фалкерсон, Эдмондс-Карп.

1. Введение

Сеть – ориентированный граф $G = (V, E)$, в котором выделены две вершины: **исток** s и **сток** t. Каждому ребру (u, v) приписана неотрицательная **пропускная способность** $c(u, v)$. Если ребра (u, v) нет в графе, считаем $c(u, v) = 0$.

Поток – функция $f: V \times V \to R$, удовлетворяющая:

• **Ограниченность пропускной способностью**: $0 \leq f(u, v) \leq c(u, v)$ для всех u, v.

• **Закон сохранения**: для любой вершины $v \notin \{s, t\}$ сумма потоков, входящих в v, равна сумме выходящих потоков.

• **Антисимметричность**: $f(u, v) = -f(v, u)$.

Величина потока $|f|$ – сумма потоков, выходящих из истока (или, что то же самое, входящих в сток).

Задача: найти поток максимальной величины.

2. Основные понятия

2.1 Остаточная сеть

Для заданного потока f определим **остаточную пропускную способность** ребра (u, v):

$$c_f(u, v) = c(u, v) - f(u, v).$$

Остаточная сеть G_f содержит те же вершины, что и G, и все рёбра (u, v), для которых $c_f(u, v) > 0$. Если $f(u, v) > 0$, то в G_f появляется также обратное ребро (v, u) с остаточной пропускной способностью $f(u, v)$ (поскольку уменьшение потока равносильно увеличению по обратному направлению).

2.2 Дополняющий путь

Дополняющий путь в остаточной сети – простой путь из s в t, все рёбра которого имеют положительную остаточную пропускную способность. Вдоль такого пути можно «пропустить» дополнительный поток величиной, равной минимуму остаточных способностей на рёбрах пути.

Теорема (основная): поток f максимален **тогда и только тогда**, когда в остаточной сети G_f нет дополняющего пути.

2.3 Декомпозиция потока

Любой поток можно разложить в сумму потоков вдоль путей из s в t и циклов (с сохранением ограничений пропускных способностей). Это свойство часто используется при доказательствах и построении алгоритмов.

3. Алгоритм Форда-Фалкерсона

Идея: начинаем с нулевого потока. Пока в остаточной сети существует дополняющий путь, увеличиваем поток вдоль него на максимально возможную величину (минимальную остаточную пропускную способность на пути).

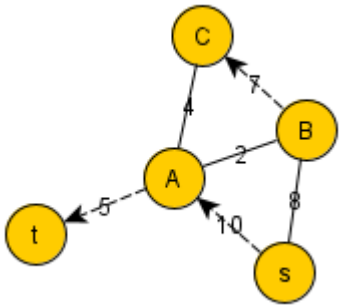
3.1 Псевдокод (поиск пути с помощью DFS)

```
function ford_fulkerson(G, s, t, c):
   for each edge (u, v) in G:
      f[u][v] = 0
      f[v][u] = 0                    // для антисимметричности
   max_flow = 0
   while true:
      # Поиск дополняющего пути в остаточной сети с помощью DFS
      path, min_cap = dfs_find_path(s, t, INF)
      if min_cap == 0:               // пути нет
         break
      max_flow += min_cap
      # Обновляем поток вдоль найденного пути
      for each edge (u, v) in path:
         f[u][v] += min_cap
         f[v][u] -= min_cap
   return max_flow

function dfs_find_path(u, t, min_cap):
   if u == t:
      return [t], min_cap
   mark u as visited
   for each v such that c[u][v] - f[u][v] > 0:
      if not visited[v]:
         path, flow = dfs_find_path(v, t, min(min_cap, c[u][v] - f[u][v]))
         if flow > 0:
            prepend u to path
            return path, flow
   return [], 0
```

3.2 Пример работы

Рассмотрим сеть:



Пусть сначала поток нулевой. Остаточная сеть совпадает с исходной. DFS может найти путь $s \to A \to t$ с пропускной способностью $min(10, 5) = 5$. Увеличим поток на 5. Теперь ребро $A \to t$ насыщено, в остаточной сети появляется обратное ребро $t \to A$ с пропускной способностью 5.

На следующем шаге DFS может найти $s \to B \to C \to t$? Но рёбер $B \to C$ нет, поэтому, возможно, найдётся $s \to A \to C \to t$, если добавить ребро $A \to C$? В исходной сети его нет. Важно, что благодаря обратным рёбрам можно «перебрасывать» поток, что позволяет находить более сложные комбинации. В итоге алгоритм сойдётся к максимальному потоку (в данном примере максимальный поток равен 12).

3.3 Сложность и недостатки

Каждое увеличение потока требует DFS – $O(E)$. Число увеличений может быть равно величине максимального потока (если все пропускные способности целые, и каждый раз увеличиваем на 1). Время работы: $O(|f_{max}| \cdot E)$, что не является полиномом от размера графа. На практике может работать очень медленно, если пропускные способности велики.

4. Алгоритм Эдмондса-Карпа

Улучшение: вместо произвольного поиска (DFS) использовать поиск кратчайшего по числу рёбер дополняющего пути в остаточной сети с помощью BFS.

4.1 Псевдокод

```
function edmonds_karp(G, s, t, c):
   for each edge (u, v) in G:
      f[u][v] = 0
      f[v][u] = 0
   max_flow = 0
```

```
while true:
    # BFS в остаточной сети строит дерево кратчайших путей
    parent = array of size |V| filled with -1
    parent[s] = s
    queue = [s]
    found = false
    while queue not empty and not found:
        u = queue.pop_front()
        for each v such that c[u][v] - f[u][v] > 0:
            if parent[v] == -1:      # не посещена
                parent[v] = u
                if v == t:
                    found = true
                    break
                queue.append(v)
    if not found:                    # нет дополняющего пути
        break
    # Находим минимальную остаточную пропускную способность на пути
    path_flow = INF
    v = t
    while v != s:
        u = parent[v]
        path_flow = min(path_flow, c[u][v] - f[u][v])
        v = u
    # Обновляем поток
    v = t
    while v != s:
        u = parent[v]
        f[u][v] += path_flow
        f[v][u] -= path_flow
        v = u
    max_flow += path_flow
return max_flow
```

4.2 Обоснование эффективности

Ключевой факт: при использовании BFS расстояние (количество рёбер) от s до любой вершины в остаточной сети не убывает от одной итерации к другой. Каждое ребро может стать насыщенным (и исчезнуть из остаточной сети) не более $O(V)$ раз, поскольку каждый раз, когда оно исчезает и позже появляется снова (как обратное), расстояние до его начала увеличивается. Поэтому общее число дополняющих путей – $O(V \cdot E)$. Каждый BFS выполняется за $O(E)$. Итоговая сложность: $O(V \cdot E^2)$.

4.3 Пример, где Форд-Фалкерсон медленен

Рассмотрим граф с длинным «змеевидным» путём и маленькими обратными рёбрами, где DFS постоянно выбирает путь, увеличивающий поток на 1. Эдмондс-Карп за счёт BFS будет находить более длинные пути, но их число ограничено, и время работы остаётся полиномиальным.

5. Теорема о максимальном потоке и минимальном разрезе

5.1 Определение разреза

Разрезом (S, T) в сети называется разбиение множества вершин V на два подмножества: S (содержит s) и $T = V \setminus S$ (содержит t). **Пропускная способность разреза** – сумма пропускных способностей рёбер, идущих из S в T:

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

Теорема (Форд-Фалкерсон): для любой сети величина максимального потока равна минимальной пропускной способности разреза.

5.2 Доказательство (идея)

Из определения потока легко видеть, что любой поток $|f|$ не превосходит пропускной способности любого разреза (так как весь поток должен пересечь разрез). Обратно: если поток максимален, то в остаточной сети нет пути из s в t. Пусть S – множество вершин, достижимых из s в остаточной сети. Тогда $t \notin S$, и все рёбра из S в T в исходной сети насыщены, а обратные имеют нулевой поток. Поэтому суммарный поток через разрез равен его пропускной способности, и следовательно, $|f| = c(S, T)$.

5.3 Построение минимального разреза по максимальному потоку

После завершения работы алгоритма (любого, находящего максимальный поток) в остаточной сети запускаем BFS/DFS из s. Множество посещённых вершин – S, остальные – T. Рёбра из S в T в исходной сети и есть искомый минимальный разрез.

6. Сравнение методов

Метод	**Время работы**	**Память**	**Особенности**
Форд-Фалкерсон (DFS)	$O(\lvert f_{max} \rvert \cdot E)$	$O(V + E)$	Не полиномиален, прост
Эдмондс-Карп (BFS)	$O(V \cdot E^2)$	$O(V + E)$	Полиномиален, надёжен

Масштабирование пропускных способностей	$O(E^2 \cdot \log C)$	$O(V + E)$	Улучшение для целых больших C

(C – максимальная пропускная способность)

7. Дополнительные темы

7.1 Целочисленность потоков

Если все пропускные способности целые, то алгоритмы Форда-Фалкерсона и Эдмондса-Карпа сохраняют целочисленность потока на всех шагах (поскольку дополняющие пути увеличивают поток на целую величину). Это важно для приложений, где поток интерпретируется как количество единиц товара, маршрутов и т.д.

7.2 Применение: поиск рёберно-непересекающихся путей

Задача о максимальном количестве путей из s в t, не имеющих общих рёбер, сводится к построению максимального потока в сети с единичными пропускными способностями. Максимальный поток равен числу таких путей.

7.3 Масштабирование

Идея: сначала ищем дополняющие пути с большим приращением (например, не меньше заданного порога), затем постепенно уменьшаем порог. Это даёт полиномиальную сложность $O(E^2 \cdot \log C)$, где C – максимальная пропускная способность.